



NATIONAL UNIVERSITY
of Computer & Emerging Sciences

Operating Systems

Final Project Report

Team Members:

- Umar Asghar (i242532)
- Wasil Kiyani (i242551)
- Dawar Ahmed (i242533)

Submitted To: Ma'am Kanza, Sir Hasan Mujtaba

Class: DS

Section: 4B

Submission Date: May 3rd, 2026

Section 2: Problem Statement

Most simple computer programs are slow because they do one thing at a time. For example, if a program has to read a huge file with sensor data, it stops everything else just to read a single line. This wastes the power of modern computers, which have many "cores" or "brains" that can work at the same time.

We solved this by building a "Pipeline". Instead of one program doing all the work, we split it into four smaller programs. This way, while one part is reading the file, another part is doing the math, and a third part is writing the report. This makes the whole process much faster and uses the full power of the computer.

Section 3: System Design and Architecture

3.1 How the Programs are Split

We used the Linux "Process" model to keep our programs safe and separate:

- **Dispatcher:** The "Boss" process. It starts the others and makes sure they clean up when they finish.
- **Ingestor:** The "Fetcher." It only cares about finding and reading CSV files.
- **Processor:** The "Math Engine." It uses a team of threads to calculate averages and find errors in the data.
- **Reporter:** The "Writer." it waits until the very end to save the results into a file.

3.2 How They Talk (IPC)

Since these are separate programs, they need special ways to talk:

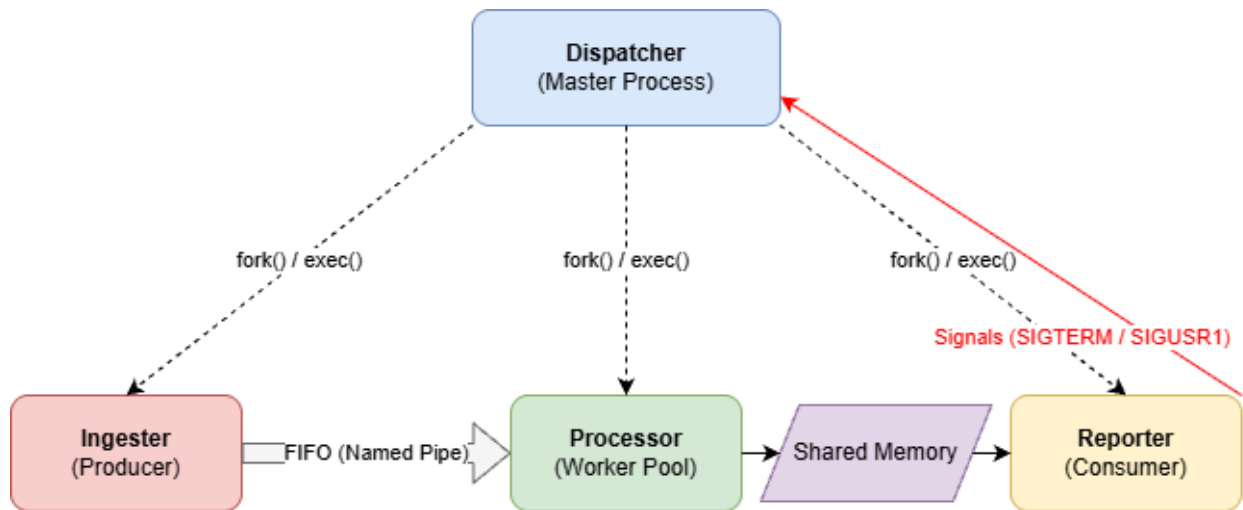
1. **Named Pipes (FIFO):** Think of this as a one-way tunnel. The Ingestor "pushes" data into one end, and the Processor "pulls" it out from the other.
2. **Shared Memory:** This is like a shared whiteboard. The Processor writes the final numbers on it, and the Reporter reads them. It is the fastest way to share big data.
3. **Semaphores:** These are like "Traffic Lights." They tell the Reporter when the math is 100% finished so it can start writing the report.

3.3 Synchronization (Staying Organized)

Inside the Processor, we use a **Bounded Buffer** (a small storage box) to hold data chunks:

- **Semaphores:** We use two lights to make sure the "box" doesn't overflow and that threads don't try to grab data from an empty box.

- **Mutex:** Only one thread can update a sensor's average at a time. The thread must "lock" the data, change it, and then "unlock" it for the next thread.



Section 4: Implementation Details

4.1 Making it Flexible

Our program is not "hardcoded." You can tell it how many threads to use or which folder to read from using the terminal. The Dispatcher takes these numbers and passes them to the children using `execvp()`. This means you can run the program on a small laptop or a giant server without changing the code.

4.2 The "Poison Pill" Shutdown

When you want to stop the program (Ctrl+C), we use a "Backward Pass". The Dispatcher sends a signal to the Ingestor. The Ingestor sends a special "End of Work" message (called a Poison Pill) into the pipe. When the threads see this, they know they can safely stop working.

4.3 Thread Choices

We chose specific settings for our worker threads:

- **Chunk Size (4KB):** We gave each thread a small amount of memory (4MB) so we could start hundreds of them without crashing the computer.
- **Joinable State:** We made the threads "Joinable" so the main program can wait for every single one to finish its work before the Reporter starts.

4.4 The `dup()` Redirect

To write our report, we used a trick. We used `dup()` to "save" the screen, then used `dup2()` to point all the program's output to a file. After the report was written, we pointed it back to the screen so the user could see a "Finished" message.

Section 5: Sample Output

```
wasil@wasil-Victus-by-HP-Gaming-Laptop-15-fb2xxx:~/Downloads/OS_FinalProject_FINAL/OS_FinalProject_Section$ make clean
killall processor 2>/dev/null || true
rm -f /tmp/test_pipe /tmp/master_pipe
rm -f dispatcher ingester processor reporter
rm -rf logs/* report.txt report.csv .dispatcher.pid
wasil@wasil-Victus-by-HP-Gaming-Laptop-15-fb2xxx:~/Downloads/OS_FinalProject_FINAL/OS_FinalProject_Section$ bash run.sh -n 4
gcc -Wall -Wextra -pthread -Isrc/common -o dispatcher src/dispatcher.c -lrt
gcc -Wall -Wextra -pthread -Isrc/common -o ingester src/ingester.c -lrt
gcc -Wall -Wextra -pthread -Isrc/common -o processor src/processor.c -lrt
gcc -Wall -Wextra -pthread -Isrc/common -o reporter src/reporter.c -lrt
[Dispatcher] Starting Dispatcher (PID: 9012, PPID: 8988)
[Dispatcher] Child PID 9013 exited with status 0. Runtime: 0 seconds.
[Dispatcher] Received SIGUSR1: Reporter has finished generating files.
[Dispatcher] Child PID 9014 exited with status 0. Runtime: 0 seconds.
[Dispatcher] Child PID 9015 exited with status 0. Runtime: 0 seconds.
[Dispatcher] Pipeline finished cleanly. Resources wiped.

=====
PIPELINE SUMMARY
Total Runtime: 00:00:00.010
Records Processed: 4
=====

--- Content of report.txt ---
[Reporter] Starting Reporter (PID: 9015, PPID: 9012)
FINAL IOT SENSOR REPORT
=====
Sensor: Temp_LivingRoom | Avg: 21.93 | Anomalies: 1
Sensor: Temp_Kitchen | Avg: 24.89 | Anomalies: 1
Sensor: Volt_Main | Avg: 120.02 | Anomalies: 35
Sensor: Volt_HVAC | Avg: 231.17 | Anomalies: 25

--- Content of report.csv ---
Sensor_ID,Moving_Average,Anomaly_Count
Temp_LivingRoom,21.93,1
Temp_Kitchen,24.89,1
Volt_Main,120.02,35
Volt_HVAC,231.17,25

wasil@wasil-Victus-by-HP-Gaming-Laptop-15-fb2xxx:~/Downloads/OS_FinalProject_FINAL/OS_FinalProject_Section$
```

Testing On other CSVs

```
[Dispatcher] Child PID 9822 exited with status 0. Runtime: 0 seconds.
[Dispatcher] Received SIGUSR1: Reporter has finished generating files.
[Dispatcher] Child PID 9823 exited with status 0. Runtime: 0 seconds.
[Dispatcher] Child PID 9824 exited with status 0. Runtime: 0 seconds.
[Dispatcher] Pipeline finished cleanly. Resources wiped.

=====
PIPELINE SUMMARY
Total Runtime: 00:00:00.009
Records Processed: 12
=====

--- Content of report.txt ---
[Reporter] Starting Reporter (PID: 9824, PPID: 9821)
FINAL IOT SENSOR REPORT
=====
Sensor: Humid_Basement | Avg: 45.69 | Anomalies: 1
Sensor: Temp_LivingRoom | Avg: 21.95 | Anomalies: 2
Sensor: Temp_Garage | Avg: 12.97 | Anomalies: 1
Sensor: Temp_Bedroom | Avg: 33.28 | Anomalies: 2
Sensor: Temp_Kitchen | Avg: 25.00 | Anomalies: 2
Sensor: Volt_Main | Avg: 120.02 | Anomalies: 70
Sensor: Volt_HVAC | Avg: 218.79 | Anomalies: 58
Sensor: Volt_Bedroom | Avg: 119.96 | Anomalies: 8
Sensor: Humidity_Attic | Avg: 45.88 | Anomalies: 0
Sensor: Temp_Outside | Avg: 15.03 | Anomalies: 0
Sensor: Temp_Freezer | Avg: -1.96 | Anomalies: 5
Sensor: Engine_Heat | Avg: 101.82 | Anomalies: 5

--- Content of report.csv ---
Sensor_ID,Moving_Average,Anomaly_Count
Humid_Basement,45.69,1
Temp_LivingRoom,21.95,2
Temp_Garage,12.97,1
Temp_Bedroom,33.28,2
Temp_Kitchen,25.00,2
Volt_Main,120.02,70
Volt_HVAC,218.79,58
Volt_Bedroom,119.96,8
Humidity_Attic,45.88,0
Temp_Outside,15.03,0
Temp_Freezer,-1.96,5
Engine_Heat,101.82,5

wasil@wasil-Victus-by-HP-Gaming-Laptop-15-fb2xxx:~/Downloads/OS_FinalProject_FINAL/OS_FinalProject_Section$
```

Section 6: Work Division

- **Dawar Ahmed:** Built the Dispatcher "Boss," set up the Shared Memory, and made sure all files are deleted at the end.
- **Umar Asghar:** Made the Ingester to find and read CSV files and wrote the run.sh script to make running the project easy.
- **Wasil Kiyani:** Set up the Processor's threads and the Reporter's file-writing logic.

Section 7: Challenges Faced

1. **Deadlocks:** Sometimes the programs would just freeze. We learned that the "Pipe" blocks if you don't open both ends correctly. We fixed this by starting the Processor first.
2. **Memory Leaks:** If the program crashed, the shared memory stayed in the computer. We fixed this by adding a "cleanup" function that deletes everything even if the program fails.
3. **Race Conditions:** Threads were overwriting each other's work. We used a **Mutex** to make sure only one thread writes at a time.

Section 8: Conclusion

This project taught us how to make programs work together. By using Linux tools like Pipes, Shared Memory, and Threads, we built a system that is very fast and safe. We made sure to handle errors and clean up everything when we are done.

APPENDIX A File Screenshots

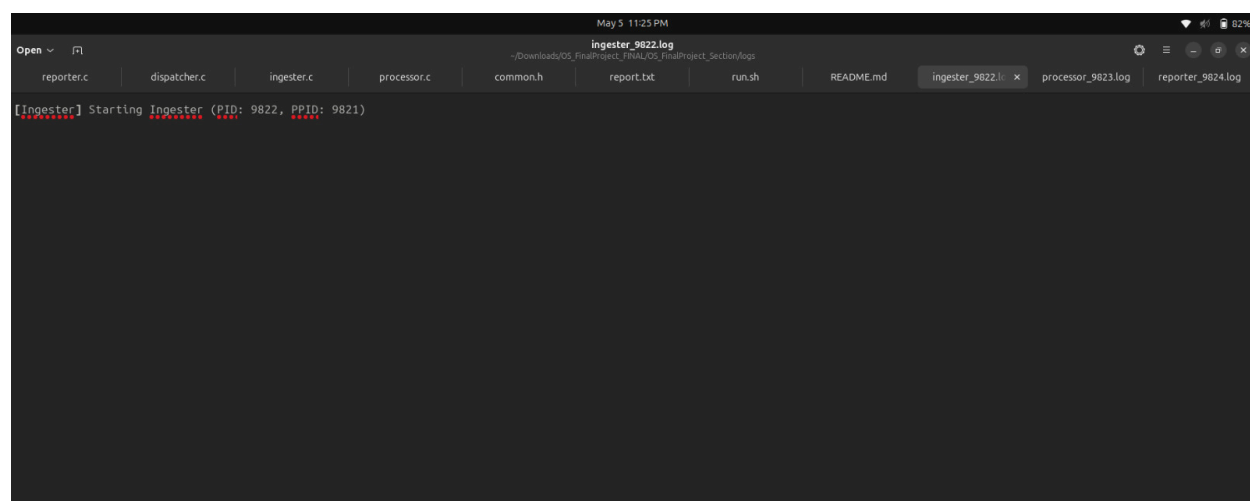
Main Folder

Name	Date modified	Type	Size
.git	5/5/2026 2:37 AM	File folder	
data	5/5/2026 2:24 AM	File folder	
logs	5/4/2026 8:31 PM	File folder	
report	5/4/2026 8:31 PM	File folder	
src	5/5/2026 2:24 AM	File folder	
DECLARATION	5/5/2026 2:24 AM	Text Document	1 KB
Makefile	5/5/2026 2:24 AM	File	1 KB
README	5/5/2026 2:24 AM	Markdown Source ...	4 KB
run	5/5/2026 2:24 AM	SH Source File	3 KB

Source Folder

Name	Date modified	Type	Size
 common	5/5/2026 2:24 AM	File folder	
 dispatcher	5/5/2026 10:03 PM	C Source File	9 KB
 ingester	5/5/2026 9:26 PM	C Source File	4 KB
 processor	5/5/2026 2:24 AM	C Source File	8 KB
 reporter	5/5/2026 2:24 AM	C Source File	4 KB

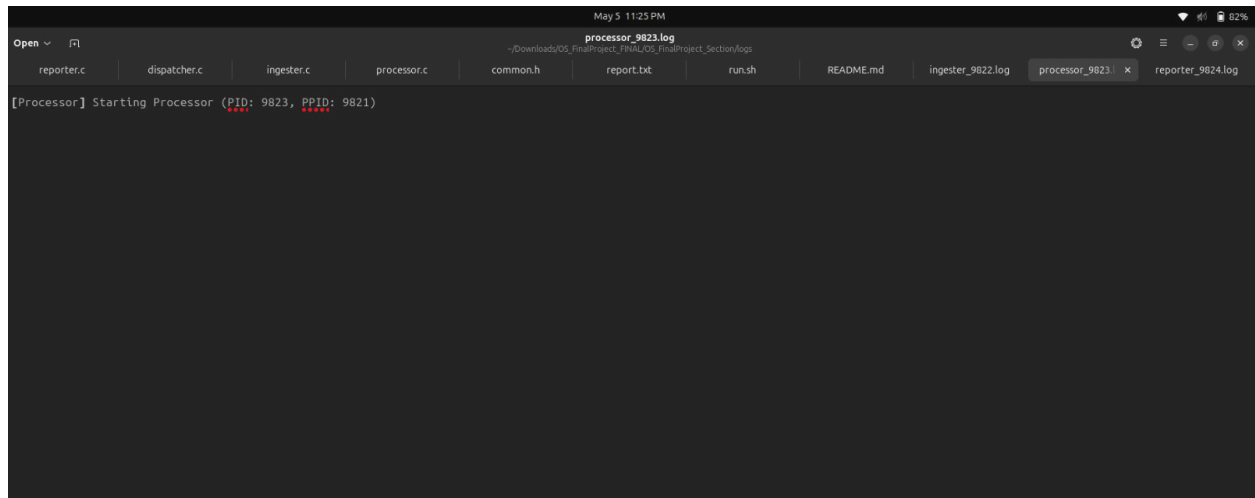
Ingester Logs



```
May 5 11:25 PM
ingester_9822.log
~/Downloads/OS_FinalProject_FINAL/OS_FinalProject_Section/Logs
reporter.c dispatcher.c ingester.c processor.c common.h report.txt run.sh README.md ingester_9822.log processor_9823.log reporter_9824.log

[Ingester] Starting Ingester (PID: 9822, PPID: 9821)
```

Process Logs



APPENDIX B Code Snippets

Standard Libraries Used

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <fcntl.h>
#include <string.h>
#include <signal.h>
#include <dirent.h>
#include <sys/wait.h>
#include <sys/stat.h>
#include <sys/mman.h>
#include <semaphore.h>
#include <pthread.h>
#include <time.h>
```


Agreed Limits And Constrains

```
#define MAX_CHUNK_SIZE 1024    //Max bytes per chunk

#define MAX_SENSORS 50         //Max unique IoT devices we expect

#define MAX_SENSOR_NAME 32     //Max characters in a sensor's name


//3. THE PIPE (Ingester -> Processor)

typedef struct {

    int chunk_id;

    size_t byte_count;

    int source_file_id;

} ChunkHeader;


//4. THE SHARED MEMORY (Processor -> Reporter)

typedef struct {

    char sensor_id[MAX_SENSOR_NAME];

    double moving_average;

    int anomaly_count;

} IoTAggregationRecord;
```

```
typedef struct {  
  
    int unique_sensor_count;  
  
    IoTAggregationRecord records[MAX_SENSORS];  
} SharedData;  
  
#endif
```